# Implementation and Evaluation of an Enhanced Intention Prediction Algorithm for Lane-Changing Scenarios on Highway Roads

Omar Laimona*†, Mohamed A. Manzour*†, Omar M. Shehata*†‡, and Elsayed I. Morgan*†
*Multi-Robot Systems (MRS) Research Group, Cairo, Egypt
†Mechatronics Department - Faculty of Engineering and Materials Science - German University in Cairo, Egypt
‡Mechatronics Department - Faculty of Engineering - Ain Shams University Cairo, Egypt
Email: mrs.research.lab@gmail.com, laymonaomar@gmail.com, mohamed.manzour@ieee.org,
omar.shehata@ieee.org, elsayed.morgan@guc.edu.eg

***Abstract*—For an autonomous vehicle driving on a public road, the safety of the passengers and the efficiency of the trip taken are prioritized causing the main function of the autonomous vehicle to be interpreting and inferring the intention of surrounding vehicles, and warning the driver accordingly. Recent Advanced Driving Assistance Systems (ADAS) are capable of and usually limited to, support features like forward-collision warnings, alerting the driver of hazardous road conditions, detecting road markings, and warning the driver if they are changing lanes. However, modern ADAS are still unable to perform basic vehicle-behavior-prediction humans are capable of. In this paper, we introduce and compare the results of two different methodologies, Recurrent Neural Networks (RNN) and Long Short-Term Memory networks (LSTM), for lane-changing intention prediction of surrounding vehicles. For the LSTM model, the F1-score achieved was 0.944 for lane-keeping, 0.781 for left lane-changing, and 0.942 for right lane-changing. The RNN-based model reached an F1-score of 0.704 for lane-keeping, 0.533 for left lane-changing, and 0.714 for right lane-changing. The training process of these data-driven based methodologies can be implemented using sequences of changing centroids of vehicles along with the frames and labeling of the maneuvers introduced by the PREVENTION dataset.**

***Index Terms*—Intention Prediction; Lane-Changing; RNN; LSTM.**

## I. Introduction and Related Works

With the fast-growing inter-dependency between nations, organizations, and people in general, it is required by humans to be traveling from one place to another due to various reasons. This resulted in humans spending a large portion of their time traveling. As indicated by [1], an average American spends 48.4 minutes driving daily, which translates to an average of 294 hours of driving annually. With today's average traffic and transportation systems, safety is far from guaranteed and so when the time spent in cars increases, the number of accidents increases. In, Egypt a report was done by the World Health Organization (WHO) indicating that the number of deaths that occurred due to traffic accidents was about 12,000 annually with 42 deaths per 100,000 populations [2]. The majority of accidents occur on highway roads and one of the common contributors to these accidents is improper lane-changing maneuvers as indicated by research [3]. Fully autonomous vehicles with no predictive abilities have to act in a very conservative fashion, as stable as possible, in the existence of other vehicles surrounding it. Developing a dependable algorithm for intention prediction of surrounding vehicles is essential for safe and efficient autonomous driving. In this paper, we adopt a data-driven learn-based approach in which a significant amount of data will be needed and so the PREVENTION dataset [4] was chosen to be utilized in our research purpose.

Some models have been presented for predicting the intentions of different traffic participants. Survey [5], a collection of these models is provided. One of these models is the physics-based motion model. The physics-based motion model uses either kinematic or dynamic models to describe and predict the motion of vehicles. This type of approach assumes constant speed and orientation of the vehicle and uses parameters like speed, acceleration, and heading to represent the vehicle state. Afterward, this model can be developed and used to make predictions. Several popular existing methods including the Monte Carlo simulations [6], and the constant turn rate and velocity approach [7] employ the physics-based motion model. However, this type of model is usually limited to predictions under one second [5].

Other models used in [8] [9] [10] are called data-driven based models which can learn complex motion scenarios through feeding the model with many data points such as trajectories, or sequences of changing centroids, and adjusting some hyper-parameters such as the learning rate, and dropout coefficient to be finely tuned for best model performance. Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) are examples of data-driven based models.

Several techniques that have been tested for intention prediction include using LSTM networks [8], where researchers used tracks of the vehicles to predict maneuver classes and future trajectories. In [9], researchers used representations of trajectories of the surrounding traffic participants together with a Convolutional Neural Network (CNN) to predict lane-changing intention.

Another technique for lane-changing intention prediction

978-1-7281-8226-1/20/$31.00 ©2020 IEEE

was defined by [10], in which researchers compared 2 different approaches, a CNN-based approach using image sequences which consisted of single frames, and an LSTM-based approach [11] utilizing a Region of Interest (ROI) selection method to generate bounding boxes and analyze each vehicle individually, and a GoogleNet network [12] to generate a feature vector for each image. These researchers also used the PREVENTION dataset to predict lane-changing intention. While using the CNN-based approach, the training data consisted of images, in which features like the past 10 contours of the vehicles, and the centers of these contours were used. However, while using the LSTM-based approach, it was found that the image sizes were too huge to train the network on and so a 1024x1 feature vector was produced for each image containing the attributes mentioned before. When comparing these two approaches it was indicated that the LSTM-based approach had an advantage over the CNN-based approach achieving a recall of 61.7% percent for lane-changing, 89.3% for left lane-changing, and 68.6% for right lane-changing.

In this paper, we are interested in RNN and LSTM data-driven based methods to classify the intention of other vehicles into three classes which are lane-keeping, left lane-changing, and right lane-changing. The input given to the model is extracted from the PREVENTION dataset. The input is a sequence of changing centroids of the ego-vehicle along with the frames.

The rest of the paper is organized as follows. Section II describes the PREVENTION dataset used and the methodology behind the extraction of the data as well as preparation and analysis of the features used. Section III introduces the RNN and the LSTM-based approaches developed for lane-changing intention prediction using sequences of changing centroids of vehicles along with the frames. Section IV discusses the results obtained by the two models proposed in section III. Finally, Section V provides a conclusion for the paper together with some recommended future work.

## II. Methodology-Dataset

The data used in this thesis is the PREVENTION dataset, which is a very recent dataset collected in 2019 by members of the Computer Engineering Department, Universidad de Alcala, Alcala de Henares, Spain. The PREVENTION dataset is publicly available at *https://prevention-dataset.uah.es* and contains 356 minutes of total recordings consisting of 540 km of distance traveled by a vehicle on a highway. For our purpose, only visual information obtained from the camera sensors readings will be considered. The camera sensors used were specifically two Grasshopper3 cameras, mounting 12.5 mm fixed focal length lens, and they were covering a Field Of View (FOV) of 48 degrees one in the front and the other in the back.

### A. Labeling and Filtering

First, a top-level segmentation is performed on the images, differentiating between *cars*, *trucks*, *buses*, *motorcycles*, *bicycles*, and *pedestrians* with a label for each category. For our research purpose, we will focus only on vehicles, so all other detections are not taken into account. Raw output detections are provided in the form of contours that are temporally integrated meaning that each detection is provided with the frame at which the contour was retrieved. Furthermore, only raw detections with a minimum confidence value of 0.5 are taken into consideration while ruling out all other detections.

Upon accessing the dataset's website, five recordings were found each divided into *n* DRIVES and each **DRIVE** consists of calibration information, and subsections of raw data, which are the raw videos, and processed data. *processed_data* contains *detection_cameraN* folders, which consists of *detections.txt*, that presents all detections with bounding boxes and contours of all objects in the visual scene. *detections_filtered.txt* is a sub-file of *detections.txt* in which filtering of data occurred so that it includes data with a confidence value of 0.5 or more. *detections_tracked.txt* is a sub-file of *detections_filtered.txt* where tracking of the filtered detections occurred so that each object is provided with a unique ID along the frames and data is stored in the form of [*frame*, *ID*, *class*, *xi*, *yi*, *xf*, *yf*, *conf*, *n*] where *class* is the variable used to differentiate between the types of vehicles tracked, and *n* is a sequence of *x* and *y* coordinates defining the contour of the tracked vehicle.

*lane_changes.txt* is a file that contains the information regarding the detected lane-changes of different vehicles and is arranged as [*ID*, *ID-m*, *LC-type*, *f0*, *f1*, *f2*, *blinker*] where *ID-m* is the unique ID provided for each vehicle performing the lane-changing, *LC-type* is the label for each detected lane-changing, can be left *3* or right *4*, *f0* is the frame at which lane changing started, *f1* is the frame at which the lane-changing event occurred and this is logged when the rear middle part of the vehicle is just between the lanes, *f2* is the finishing frame as illustrated by Figure 1, and *blinker* is a Boolean value which describes the detection of a blinker during the lane-changing event, (0) no blinker, (1) blinker detected. Lane-changing events and blinker detections are not always associated with each other meaning that a blinker detection can occur without a lane-changing event taking place and vise versa.

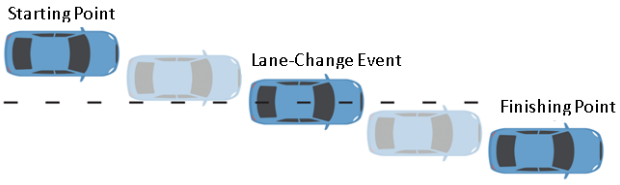
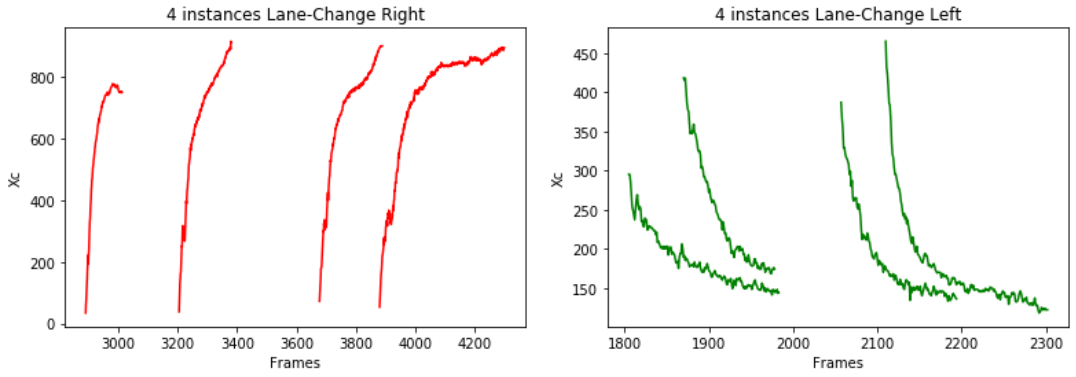



Fig. 1: Lane-changing Labeling [4]

### B. Features Extraction

The *detections_tracked*, and *lane_changes* text files described before were found very helpful to attain that target, as they had useful information like the *x* and *y* coordinates describing the contours of the vehicles together with the




(a) (b)

Fig. 2: Relationship between Frames and X coordinates of 4 vehicles performing (a) a right lane-changing, (b) a left lane-changing.

lane-changing labeling. This information can be collected, extracted, and prepared to be used for training the model.

First, the data was extracted from the text files. Using the $x$ and $y$ coordinates that describe the contour of the vehicles in a frame, the centroid of each vehicle was calculated, by averaging the total number of points describing the contour that contains the vehicle, and stored together with the ID of the vehicle. Also, the frame at which this contour was described was stored so that each vehicle was represented by its ID, center point that consisted of $x$ and $y$ center coordinates, and the frame of this occurrence in the video. Then utilizing the data provided in the lane-changes text file, sequences of features were created consisting of the ID of the vehicle performing the lane-changing event, the frames at which the event occurred starting with the starting frame (f0) and ending with the finishing frame (f2), and finally $x$ and $y$ coordinates of the centroid of the vehicle performing the lane-changing maneuver in that frame. The $x$ and $y$ coordinates together with the changing frames created a sequence that identifies the change in the center point of the target vehicle with respect to the ego vehicle. This sequence can be used to train the model eventually causing it to interpret patterns and making it able to label any other external patterns on its own. Figure 2a describes the relationship between the *frames* and $x$ coordinates of 4 vehicles performing a right lane-changing. Figure 2b describes the relationship between the *frames* and the $x$ coordinate of 4 vehicles performing a left lane-changing.

All instances in each Figure share similarly shaped curves produced by vehicles performing the same maneuver (left or right lane-changing) in different frames. The final total training data contained 292 lane-change left maneuvers, and 422 lane-change right maneuvers.

## III. Learning Model

Data-driven approaches were used due to their great performance in many applications. These approaches were implemented through two different architectures: Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM). A primitive example of a data-driven approach is Artificial Neural Networks (ANN). ANNs are computing systems that were first invented to mimic the action of biological neurons. The most basic ANN architecture is the feed-forward Network (FFN), having a building block of a neuron that takes several real-valued variables, $xi$, as inputs and outputs a real-valued variable through a series of computations shown by Eq. (1). LSTMs are an implementation of the Recurrent Neural Network (RNN), which is one type of the ANNs and differs from the basic FFNs in the way of dealing with the output of the neuron.

RNNs act as loops in which iterations occur. In each of these iterations, the output is calculated then fed back into the unit so that the output is passed forward together with the new input to the network, introducing what is called the recurrent edges. These recurrent edges changed the structure of the basic neuron creating cycles that connect the previous time-step with the next one through the output furthermore introducing the sense of time to the network. Introducing the sense of time to the network was extremely useful when training the network on data that has a strong connection between the time-steps of each sample, like sequential data collected from camera sensors as in our case.

LSTMs differ from RNNs in the way that they do not suffer from the Vanishing or Exploding Gradients problem [13], which occurs during back-propagation [14] when there are too many layers with weights that need to be updated. The gradients diminish for every layer it passes through causing the gradients to converge to zero rather quickly. However, in an LSTM neural network, there is not any exponentially, or fast-decaying factor involved during back-propagation. During back-propagation the output is multiplied by what is called the forget gate and continues to the previous state rather than multiplying by the weights that might be decaying, avoiding the vanishing gradient problem. In this article, we are going to implement both the RNN and the LSTM networks, observe

their results, and compare them.

$$output = g(\sum wixi + b) \quad (1)$$

An LSTM also acts as a loop and its unit is composed of four components that interact together. One of the main components is the cell state, which carries the information of the unit. Information can be added to or removed from the cell state by the unit and the procedure of modifying the information of the cell state is controlled by gates, which output a number between zero and one describing the amount of information to be passed on to, or removed from, the cell state, with "*0*" describing no information and "*1*" describing all information. The other three components are consisted of three gates: the forget gate, the input gate, and the output gate. The forget gate is responsible for deciding what information is not needed and is going to be removed from the cell state. The input gate is responsible for deciding what new information is going to be added to the cell state. Finally, the output gate controls the amount of information to be passed on from the cell state to the output. Due to their ability to work well with sequential data, LSTM networks would serve our research purpose effectively.

This structure of LSTMs also provides them with the capability of learning not only short-term relations between features, but it also remembers long-term relations between these features. That is why LSTMs are very powerful and useful when using time series data for training as in our case. Time series data is defined as a sequence of $n$ real-valued variables, which is formed by observing and collecting data from an underlying operation over time and can be univariate or multivariate. In univariate data one observation is accounted for, which is our case, and in multivariate data more than one observation is taken into account simultaneously.

### A. Network Architecture

From the dataset section, it is clear that the data extracted and prepared is sequential, time-series data where there is strong correlation in between the centroids of the vehicles detected in the maneuver and the frames at which the detection occurred. For that reason, RNNs and LSTM neural network are very well suited for this type of data.

*1) RNN Architecture:* The number of layers of Simple RNNs used is 2. The first layer is composed of 512 units and the second contained 256 units, with the relu activation function in the input layer. Also, a Dropout layer [15] after each layer is used with a value of 0.2, and finally a Dense layer, containing 3 units and a softmax activation function that produces the probability along with each predicted class.

*2) LSTM Architecture:* The number of layers of LSTMs is 2 with the first containing 512 units and the second one containing 256 units, selected carefully to balance the models performance together with the computational effort. In the input layer the, "relu" activation function is used and after each LSTM layer a Dropout layer with the value of 0.5 is used and finally a Dense layer with 3 units having a softmax activation function.

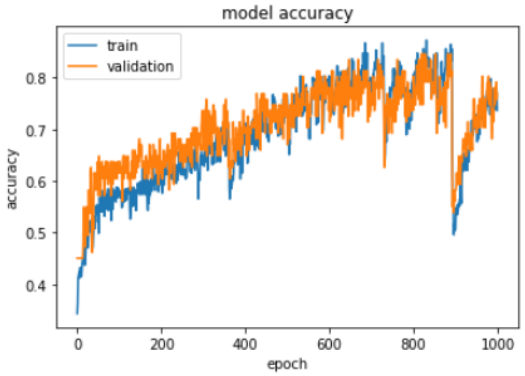
### B. Input Features

For better apprehension of the models towards the strong correlation found between the changing centroids of the vehicles along with the frames, the network will be fed only the sequences of centroids containing the $x$ coordinates as well as the $y$ coordinates together with each sequence label. Each sequence starts with the starting frame, where the vehicle enters the camera's field and has a length of 50 frames.

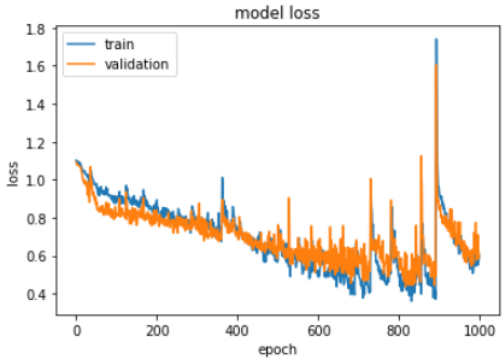
### C. Training and Evaluation

During the training process, the training data was divided into 80% training and 20% for validation. The number of samples input during each pass over the data, known as the batch size, was set to a mini-batch size of 150 for the RNN architecture and 130 for the LSTM architecture. Both models were trained for 1000 epochs, using the adam optimizer [16] with the learning rate set to 0.0005, the metrics were set to accuracy, and the process took approximately 31 minutes for the LSTM network and 2 minutes for the RNN network on a Intel® Core™ i5-5200U CPU @ 2.20GHZ with 4.00 GB RAM.

The results shown in the tables below have shown that the LSTMs outperformed the RNNs in all the experiments, and so we are going to observe the learning procedure of the LSTM neural network in terms of model accuracy and loss shown in Figures 3a and 3b below.




(a)




(b)

Fig. 3: (a) Model Accuracy During Training, (b) Model Loss During Training

## IV. Experiments and Results

Table I shows the recall, precision, and calculated F1-score from testing both models on each of the classes, lane-keeping, left lane-changing and right lane-changing.

TABLE I: RNN and LSTM Tested Recall

| | RNN | | | LSTM | | |
|---|---|---|---|---|---|---|
| | none | left | right | none | left | right |
| Recall | 76.6% | 50% | 66.6% | **96.6%** | **85%** | **93.3%** |
| Precision | 65.2% | 57.1% | 76.9% | **92.3%** | **72.2%** | **95.2%** |
| F1-score | 0.704 | 0.533 | 0.714 | **0.944** | **0.781** | **0.942** |

As shown in the table I, the LSTM model had a significant advantage over the RNN model and that verifies the vanishing gradient problem discussed before. For comparison between the performance reached for predicting each lane-changing maneuver, it was observed that the models achieved lower performance when trying to predict the left lane-change maneuver. This could be justified due to the smaller number of data entries the dataset had for lane-change left having 292 entries. However, for lane-change right the dataset contained 422 data entries. Also, the LSTM architecture has exceeded the performance reached by the models used in [10] by far, which is due to the lower length of sequences used (10), while a sequence length of 50 was used in this work, and for training the models for a reduced number of epochs (16), while in this article the models were trained using 1000 epochs allowing them to grasp the relationship between the changing centroids and the labels of the maneuvers.

Testing different model parameters and structures was of major importance. As to develop models that retrieve the best accuracy and loss, fine tuning of the parameters was needed. The procedure followed was to tune the parameters of the model to be trained and tested on the dataset. F1-score was presented for each class, lane-keeping, left lane-changing and right lane-changing. The parameters that we tested were the learning rate used by the optimizer, the coefficients in the dropout layers, and the sequence length.

### A. Learning Rate

The learning rate is a hyperparameter in the optimizer that is used to search for the local minimum in the loss function in order to update the weights accordingly. Choosing the correct learning rate would lead to a robust training process and if the learning rate is too small this might lead to a long training procedure and may cause the model to be stuck in the local minima and not learn at all, while using a large learning rate may result in increasing the training error causing an unstable training process.

In this section, we will test the effect of 3 different-valued learning rates while fixing all the other parameters. Post experimentation was carried out in order to determine the range of learning rates that would best fit our training process and it was found that learning rates between 0.0001 and 0.001 retrieve the best model performance. Therefore the learning rates that are going to be tested are 0.0001, 0.0005, and 0.001. The effect of each learning rate is going to be observed on the F1-score calculated when each class is tested after the training process. Table II shows the results.

TABLE II: Calculated F1-Scores Showing The Effect of Using Different Learning Rates on Models

| | RNN | | | LSTM | | |
|---|---|---|---|---|---|---|
| Learning Rate | none | left | right | none | left | right |
| 0.0001 | 0.682 | 0.515 | 0.710 | **0.909** | **0.753** | **0.886** |
| 0.0005 | 0.704 | 0.533 | 0.714 | **0.944** | **0.781** | **0.942** |
| 0.001 | 0.674 | 0.551 | 0.689 | **0.905** | **0.786** | **0.813** |

Testing each of the models using different learning rates revealed that the learning rate of 0.0005 caused the best overall F1-score, outperforming the other two learning rates.

### B. Dropout

Preventing over-fitting of a model, which is the case when the model starts learning the patterns of the dataset and is not able to generalize, is crucial in any deep learning experiment. Dropout was added after each LSTM and RNN layer. To evaluate which dropout coefficient should be utilized, 3 different coefficients will be used: none, 50%, and 90%, with none representing one extreme, 90% representing the other, and 50% representing the average in between. Also, the F1-score of each model will be calculated for each class label. Table III shows and compares the results obtained.

TABLE III: Calculated F1-Scores Showing The Effect of Using Different Dropout Coefficients on Models

| | RNN | | | LSTM | | |
|---|---|---|---|---|---|---|
| Dropout Coeff. | none | left | right | none | left | right |
| none | 0.685 | 0.527 | 0.698 | **0.859** | **0.790** | **0.899** |
| 0.5 | 0.704 | 0.533 | 0.714 | **0.944** | **0.781** | **0.942** |
| 0.9 | 0.615 | 0.510 | 0.642 | **0.870** | **0.769** | **0.886** |

Results show that using a dropout coefficient of 0.5 was an optimum to training both the RNN and the LSTM networks as they outperformed the other coefficients remarkably.

### C. Sequence Length

As noted earlier, using a larger sequence length than the length used in [10] to train the models resulted in a much larger F1-score. So in this paper, we decided to experiment the issue further on the models using different sequence lengths. The problem that we faced was due to the fact that as the sequence length increased, the number of data points retrieved from the dataset decreased, which causes the reduction in the model performance during training. For our purpose we are going to test our models using from 10 to 50 sequence lengths with an incrementation of 10 frames between each experiment and the other, with each sequence starting from the starting frame of the lane-change and having the length defined. Table IV documents the results.

TABLE IV: Calculated F1-Scores Showing The Effect of Using Different Sequence Lengths on Models

| | RNN | | | LSTM | | |
|---|---|---|---|---|---|---|
| Sequence | none | left | right | none | left | right |
| 10 | **0.615** | **0.659** | **0.751** | 0.510 | 0.531 | 0.600 |
| 20 | 0.661 | **0.895** | **0.796** | **0.701** | 0.604 | 0.630 |
| 30 | 0.739 | 0.592 | 0.516 | **0.806** | **0.748** | **0.805** |
| 40 | 0.655 | 0.742 | 0.615 | **0.910** | **0.769** | **0.830** |
| 50 | 0.704 | 0.533 | 0.714 | **0.944** | **0.781** | **0.942** |

Results show that, for the LSTM network, as the sequence length increase, the F1-score for each class increase. However, for the RNN network, as the sequence length increase, the F1-score decrease, which is justified by the vanishing gradient problem. As shown, the performance of the RNN model was better than the LSTM model when using a short length of sequence like in 10 and 20 sequence lengths. On the other hand, when the sequence length increased, the LSTM model started outperforming the RNN model like in sequence lengths 30, 40, and 50. Further increase of the sequence length was found to retrieve a very small number of data points that would not be enough to train the network on.

## V. Conclusion and Future Recommendation

In this article we proposed two different methodologies, a RNN-based model, and a LSTM network architecture to infer lane-changing intentions on a highway road using the centroids of the past contours of vehicles as input from the very recent PREVENTION dataset that provides naturalistic driving data. For the LSTM model the F1-score achieved was 0.944 for lane-keeping, 0.781 for left lane-changing, and 0.942 for right lane-changing. The RNN-based model achieved an F1-score of 0.704 for lane-keeping, 0.533 for left lane-changing, and 0.714 for right lane-changing.

To compare the two methodologies proposed to predict lane-changing intentions, the LSTM based method achieved higher performance than the RNN model. One of the main reasons that could explain that is the vanishing gradient problem.

Experimenting with the learning rate showed that using a learning rate of 0.0005 resulted in higher performance than the others. Then testing different dropout ratios presented that using a dropout ratio 0f 0.5 was optimum for our models. Finally, using different sequence lengths showed that the higher the length, the higher the F1-score calculated provided that there is enough data entries to use fro training.

Although the work presented in this article is highly primitive with some limitations, we believe that with these basic input features, these results lay down some very promising groundwork to lane-changing intention prediction for surrounding vehicles. Some limitations include the use of basic features, like the coordinates of the centroids of the surrounding vehicles, and not being able to generalize and test our model over other highways. For future recommendations, more features should be accessed and put into consideration, then an in-depth investigation should be done to decide which features are most relevant to our research purpose. Future work also include generalizing our model on several highways.

## References

[1] T. Triplett, R. Santos, S. Rosenbloom, and B. Tefft, "American driving survey: 2014–2015," 2016.

[2] W. H. Organization *et al.*, "Egypt: a national decade of action for road safety 2011-2020," Tech. Rep., 2011.

[3] J. J. Rolison, S. Regev, S. Moutari, and A. Feeney, "What are the factors that contribute to road accidents? an assessment of law enforcement views, ordinary drivers' opinions, and road accident records," *Accident Analysis & Prevention*, vol. 115, pp. 11–24, 2018.

[4] R. Izquierdo, A. Quintanar, I. Parra, D. Fernández-Llorca, and M. A. Sotelo, "The prevention dataset: a novel benchmark for prediction of vehicles intentions," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, Oct 2019, pp. 3114–3121.

[5] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *ROBOMECH journal*, vol. 1, no. 1, pp. 1–14, 2014.

[6] M. Althoff and A. Mergel, "Comparison of markov chain abstraction and monte carlo simulation for the safety assessment of autonomous cars," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 4, pp. 1237–1247, 2011.

[7] M. Roth, G. Hendeby, and F. Gustafsson, "Ekf/ukf maneuvering target tracking using coordinated turn models with polar/cartesian velocity," in *17th International Conference on Information Fusion (FUSION)*. IEEE, 2014, pp. 1–8.

[8] F. Altché and A. de La Fortelle, "An lstm network for highway trajectory prediction," in *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2017, pp. 353–359.

[9] D. Lee, Y. P. Kwon, S. McMains, and J. K. Hedrick, "Convolution neural network-based lane change intention prediction of surrounding vehicles for acc," in *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2017, pp. 1–6.

[10] R. Izquierdo, A. Quintanar, I. Parra, D. Fernández-Llorca, and M. Sotelo, "Experimental validation of lane-change intention prediction methodologies based on cnn and lstm," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2019, pp. 3657–3662.

[11] A. Graves, "Long short-term memory," in *Supervised sequence labelling with recurrent neural networks*. Springer, 2012, pp. 37–45.

[12] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.

[13] B. Hanin, "Which neural net architectures give rise to exploding and vanishing gradients?" in *Advances in Neural Information Processing Systems*, 2018, pp. 582–591.

[14] J. Guo, "Backpropagation through time," *Unpubl. ms., Harbin Institute of Technology*, vol. 40, 2013.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.